

Python 教材

新哲文院算法社 2026-2027 学年 Python 教材

适用对象：零基础学员，从安装到竞赛进阶

教材定位：课堂讲义 + 自学参考

配套文件：

Python教材_答案与解析.md

教材结构说明

本教材分为两大模块：

模块	内容	目标
基础掌握	环境搭建 → 语法基础 → 数据结构 → 函数 → 文件与异常	掌握 Python 编程基本功
竞赛进阶	排序算法 → 搜索算法 → 递归与递推	具备算法竞赛入门能力

第一部分：基础掌握

第一单元：语言基础与环境搭建

1.1 编程语言概述

什么是编程语言

编程语言是人与计算机沟通的"桥梁"。计算机只能理解二进制（0 和 1），而人类需要一种更直观的方式来下达指令——这就是编程语言的作用。

编译型 vs 解释型

类型	原理	代表语言	优点	缺点
编译型	源代码一次性翻译为机器码，生成可执行文件	C、C++	运行速度快	编译耗时，跨平台需重新编译
解释型	逐行读取源代码，边翻译边执行	Python、JavaScript	跨平台性好，开发灵活	运行速度相对较慢

Python 是解释型语言：你写的代码由 Python 解释器逐行读取并执行，无需提前编译。这也是为什么 Python 被称为"脚本语言"的原因之一。

为什么选择 Python

- **语法简洁：**接近自然语言，入门门槛低
- **生态丰富：**海量第三方库（科学计算、AI、Web 开发等）
- **应用广泛：**数据科学、人工智能、自动化运维、算法竞赛
- **社区活跃：**遇到问题容易找到解决方案

1.2 开发环境配置

安装 Python 解释器

步骤（Windows 系统）：

1. 打开浏览器，访问 Python 官网：<https://www.python.org/downloads/>
2. 点击 **Downloads**，下载最新稳定版（推荐 Python 3.10+）
3. 运行安装包，**务必勾选** **Add Python to PATH**（环境变量自动配置）
4. 选择 **Install Now**（默认安装）或 **Customize installation**（自定义路径）
5. 等待安装完成

验证安装：

6. 按 Win + R，输入 cmd，打开命令提示符
7. 输入 python --version，出现版本号即安装成功
8. 输入 python 进入交互模式，可直接输入代码并运行

Bash

```
1 C:\> python --version
2 Python 3.13.2
3
4 C:\> python
5 >>> print("Hello World")
6 Hello World
7 >>> exit()
```

⚠ 注意：切勿安装 Python 2.x，它已停止维护，语法与 Python 3 不兼容。

安装 Visual Studio Code (VSCode)

VSCode 是微软推出的轻量级代码编辑器，配合插件可以实现强大的开发功能。

安装步骤：

1. 访问官网：<https://code.visualstudio.com/> 下载安装包
2. 运行安装程序，安装路径建议全英文、无空格
3. 安装时勾选"添加到 PATH"和"右键菜单集成"

配置 Python 开发环境：

步骤	操作	说明
① 安装中文插件	Ctrl+Shift+X 打开扩展面板 → 搜索 Chinese → 安装	界面汉化
② 安装 Python 插件	扩展面板搜索 Python → 安装 Microsoft 官方插件	代码补全、调试、运行
③ 选择解释器	Ctrl+Shift+P → 输入 Python: Select Interpreter → 选择已安装的 Python	告诉 VSCode 用哪个 Python
④ 创建第一个文件	新建文件 hello.py，输入代码	必须以 .py 结尾

运行代码：

- 右键编辑器 → 选择"在终端中运行 Python 文件"
- 或按快捷键 Ctrl + F5（运行不调试）

实用快捷键：

快捷键	功能
Shift + Alt + F	格式化代码
Ctrl + /	快速注释 / 取消注释
F5	启动调试
Ctrl + Shift + P	全局命令面板

 参考教程：

Python 安装与 VSCode 配置：https://blog.csdn.net/2401_82819685/article/details/146186751

C++ 环境配置（对比参考）：https://blog.csdn.net/qq_45807140/article/details/112862592

1.3 第一个程序：Hello World

编写与运行

在 VSCode 中创建文件 `hello.py`：

Python

```
1 # 我的第一个 Python 程序
2 print("Hello World")
```

点击"在终端中运行"，输出：

Plain Text

```
1 Hello World
```

代码解析

代码	作用
# 我的第一个 Python 程序	注释，解释器忽略此行
print("Hello World")	输出函数，将内容打印到屏幕

输入输出函数详解

input() 函数——获取用户输入

Python

```
1 # 基本语法
2 variable = input([prompt])
3
4 # prompt: 可选参数，显示提示信息
5 # 返回值: 永远是字符串类型
```

Python

```
1 # 示例1: 基本输入
2 name = input()           # 用户输入 "Alice", name 的值为 "Alice"
3
4 # 示例2: 带提示的输入
5 age = input("请输入你的年龄: ") # 显示提示信息
6
7 # 示例3: 输入后转换类型 (⚠ 重要!)
8 age = int(input("请输入年龄: ")) # 将字符串转为整数
9 height = float(input("请输入身高: ")) # 将字符串转为浮点数
```

 **坑点提醒:** input() 永远返回字符串! 如果要做数学运算, 必须用 int() 或 float() 转换。

print() 函数——输出内容到屏幕

Python

```
1 # 基本语法
2 print(*objects, sep=' ', end='\n')
3
4 # *objects: 要输出的对象, 可以是多个
5 # sep: 分隔符, 默认为空格
6 # end: 结束符, 默认为换行符 \n
```

Python

```
1 # 示例1: 基本输出
2 print("Hello World")           # 输出: Hello World
3
4 # 示例2: 输出多个值
5 a, b = 10, 20
6 print(a, b, a + b)             # 输出: 10 20 30
7
8 # 示例3: 修改分隔符
9 print(1, 2, 3, sep='-')        # 输出: 1-2-3
10
11 # 示例4: 修改结束符 (不换行)
12 print("Hello", end='')
13 print("World")                 # 输出: Hello World (在同一行)
14
15 # 示例5: 格式化输出 (f-string, 推荐!)
16 name = "Alice"
17 age = 20
18 print(f"姓名: {name}, 年龄: {age}") # 输出: 姓名: Alice, 年龄: 20
```

单元练习 1.1

练习: 编写一个程序, 要求用户输入姓名和年龄, 然后输出一句完整的介绍。

输入示例:

请输入姓名: 张三

请输入年龄: 16

输出示例:

你好, 我叫张三, 今年16岁。

第二单元：变量、数据类型与运算符

2.1 变量与常量

变量的声明与赋值

Python 中变量不需要声明类型，直接赋值即可：

Python

```
1 # 变量赋值
2 x = 10          # 整数
3 name = "Alice"  # 字符串
4 pi = 3.14159    # 浮点数
5 is_student = True # 布尔值
6
7 # 多个变量同时赋值
8 a, b, c = 1, 2, 3
9
10 # 交换两个变量的值 (Python 特色写法)
11 a, b = b, a
```

命名规则与命名规范

硬性规则（必须遵守）：

规则	正确示例	错误示例
只能包含字母、数字、下划线	<code>my_var</code>	<code>my-var</code>
不能以数字开头	<code>var1</code>	<code>1var</code>
不能使用关键字	<code>my_class</code>	<code>class</code>
区分大小写	<code>Age</code> 和 <code>age</code> 不同	—

命名规范（建议遵守）：

- 蛇形命名法 (snake_case)：变量名用小写字母 + 下划线，如 `student_name`

- 见名知意: `a = 10` ❌ → `age = 10` ✅
- 常量名全大写: `MAX_SIZE = 100`

常量的定义

Python 没有真正的常量，约定俗成用全大写表示"不应被修改"的值：

Python

```
1 PI = 3.14159
2 MAX_CONNECTIONS = 100
3 GRAVITY = 9.8
```

2.2 基本数据类型

整数 (int)

Python

```
1 a = 10          # 十进制
2 b = 0b1010      # 二进制: 10
3 c = 0o12        # 八进制: 10
4 d = 0xA         # 十六进制: 10
5
6 # 整数运算
7 print(10 + 5)    # 加法: 15
8 print(10 - 5)    # 减法: 5
9 print(10 * 5)    # 乘法: 50
10 print(10 / 3)   # 除法: 3.333... (得到浮点数)
11 print(10 // 3)  # 整除: 3 (只取整数部分)
12 print(10 % 3)   # 取余: 1 (除法的余数)
13 print(10 ** 2)  # 幂运算: 100
```

浮点数 (float)

Python

```
1 a = 3.14
2 b = 2.0
3 c = 1.5e-3      # 科学计数法: 0.0015
4
5 # ⚠ 浮点数精度问题
6 print(0.1 + 0.2) # 输出: 0.30000000000000004
7 # 原因: 二进制无法精确表示某些十进制小数
8
9 # 解决办法: 使用 round() 或 decimal 模块
10 print(round(0.1 + 0.2, 2)) # 输出: 0.3
```

布尔值 (bool)

Python

```
1 # 布尔值只有两个: True 和 False
2 is_valid = True
3 is_empty = False
4
5 # 其他类型转布尔值
6 print(bool(0))      # False
7 print(bool(1))      # True
8 print(bool(""))     # False (空字符串)
9 print(bool("abc"))  # True (非空字符串)
10 print(bool([]))     # False (空列表)
11 print(bool([1,2]))  # True (非空列表)
12
13 # 布尔运算
14 print(True and False) # 与运算: False
15 print(True or False)  # 或运算: True
16 print(not True)       # 非运算: False
```

字符串 (str)

Python

```
1 # 创建字符串
2 s1 = 'hello'           # 单引号
3 s2 = "world"          # 双引号
4 s3 = '''多行
5 字符串'''             # 三引号（多行）
6
7 # 字符串操作
8 s = "hello"
9 print(len(s))          # 长度：5
10 print(s[0])            # 索引：'h'（从0开始）
11 print(s[-1])           # 负索引：'o'（从末尾开始）
12 print(s[1:4])          # 切片：'ell'（左闭右开）
13 print(s + " world")    # 连接：'hello world'
14 print(s * 3)           # 重复：'hellohellohello'
```

类型转换

Python

```
1 # 显式类型转换
2 str_num = "123"
3 num = int(str_num)      # 字符串 → 整数：123
4 float_num = float(str_num) # 字符串 → 浮点数：123.0
5 back_str = str(num)     # 整数 → 字符串："123"
6
7 # 隐式类型转换（Python 自动处理）
8 result = 3 + 2.5        # int + float → float: 5.5
```

类型检查

Python

```
1 x = 42
2 print(type(x))          # <class 'int'>
3 print(type("hi"))       # <class 'str'>
4 print(isinstance(x, int)) # True
```

2.3 运算符

算术运算符

运算符	含义	示例	结果
<code>+</code>	加法	<code>5 + 3</code>	<code>8</code>
<code>-</code>	减法	<code>5 - 3</code>	<code>2</code>
<code>*</code>	乘法	<code>5 * 3</code>	<code>15</code>
<code>/</code>	除法（浮点）	<code>5 / 3</code>	<code>1.666...</code>
<code>//</code>	整除（取商）	<code>5 // 3</code>	<code>1</code>
<code>%</code>	取余（取余数）	<code>5 % 3</code>	<code>2</code>
<code>**</code>	幂运算	<code>2 ** 10</code>	<code>1024</code>

比较运算符

运算符	含义	示例	结果
<code>==</code>	等于	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	不等于	<code>5 != 3</code>	<code>True</code>
<code>></code>	大于	<code>5 > 3</code>	<code>True</code>
<code><</code>	小于	<code>5 < 3</code>	<code>False</code>
<code>>=</code>	大于等于	<code>5 >= 5</code>	<code>True</code>
<code><=</code>	小于等于	<code>5 <= 3</code>	<code>False</code>

逻辑运算符

Python

```
1 # and (与): 两个条件都为真, 结果才为真
2 print(True and True)    # True
3 print(True and False)   # False
4
5 # or (或): 至少一个条件为真, 结果就为真
6 print(True or False)    # True
7 print(False or False)   # False
8
9 # not (非): 取反
10 print(not True)         # False
11
12 # 实际示例
13 age = 20
14 grade = 85
15
16 if age >= 18 and grade >= 60:
17     print("成年人且成绩及格")
18
19 if age < 18 or grade < 60:
20     print("未成年或成绩不及格")
```

赋值运算符与复合赋值

Python

```
1 x = 10
2 x += 5    # 等价于 x = x + 5, x 变为 15
3 x -= 3    # 等价于 x = x - 3, x 变为 12
4 x *= 2    # 等价于 x = x * 2, x 变为 24
5 x //= 4   # 等价于 x = x // 4, x 变为 6
6 x %= 5    # 等价于 x = x % 5, x 变为 1
```

运算符优先级

从高到低排列:

1. `**` (幂运算)
2. `*`, `/`, `//`, `%` (乘除)

3. `+`, `-` (加减)
4. `==`, `!=`, `<`, `>` (比较)
5. `not`
6. `and`
7. `or`

💡 建议：不确定时用括号 `()` 明确优先级，代码更清晰。

单元练习 2.1

练习1：编写一个程序，输入圆的半径（浮点数），输出圆的面积和周长（ π 取3.14159）。

输入：5

输出：面积=78.53975，周长=31.4159

练习2：编写一个程序，输入一个三位数，输出它的百位、十位和个位。

输入：365

输出：百位=3，十位=6，个位=5

第三单元：流程控制结构

3.1 条件分支

if / else / elif 结构

Python

```
1 # 单一条件
2 if condition:
3     # 条件为真时执行
4     statement1
5     statement2
6
7 # if-else 结构
8 if condition:
9     true_block
10 else:
11     false_block
12
13 # 多条件判断 (elif = else if)
14 if condition1:
15     block1
16 elif condition2:
17     block2
18 elif condition3:
19     block3
20 else:
21     else_block
```

 **缩进规则：**Python 用缩进（通常 4 个空格）来区分代码块，这是 Python 的语法要求，不是风格选择！

实例讲解

示例 1：判断奇偶数

Python

```
1 n = int(input("请输入一个整数："))
2
3 if n % 2 == 0:           # 如果 n 除以 2 的余数为 0
4     print("偶数")
5 else:
6     print("奇数")
```

执行流程分析：

Plain Text

```
1 输入 7
2 → 7 % 2 = 1 ≠ 0
3 → 进入 else 分支
4 → 输出 "奇数"
```

示例 2：成绩等级判断

Python

```
1 score = int(input("请输入成绩："))
2
3 if score >= 90:
4     print("优秀")
5 elif score >= 80:           # 隐含条件：score < 90
6     print("良好")
7 elif score >= 70:           # 隐含条件：score < 80
8     print("中等")
9 elif score >= 60:           # 隐含条件：score < 70
10    print("及格")
11 else:                       # score < 60
12    print("不及格")
```

 **技巧：**elif 是按顺序判断的，一旦某个条件满足，后面的不再检查。所以 `score >= 80` 时，实际上已经隐含了 `score < 90`。

示例 3：三元表达式（简洁写法）

Python

```
1 n = int(input())
2 result = "偶数" if n % 2 == 0 else "奇数"
3 print(result)
```

嵌套条件

Python

```
1 age = int(input("请输入年龄: "))
2 has_id = input("是否有身份证 (y/n): ")
3
4 if age >= 18:
5     if has_id == 'y':
6         print("可以进入")
7     else:
8         print("需要出示身份证")
9 else:
10    print("未成年人禁止进入")
```

单元练习 3.1

练习1：输入一个年份，判断是否为闰年。

规则：能被4整除但不能被100整除，或能被400整除的年份是闰年。

输入/输出示例：

输入：2000 → 输出：闰年

输入：1900 → 输出：平年

输入：2024 → 输出：闰年

练习2：输入三个整数，找出并输出最大值。

输入： 12 45 23

输出： 最大值是45

3.2 循环结构

while 循环

原理：当条件为真时，重复执行代码块，直到条件变为假。

Python

```
1 # 基本语法
2 while condition:
3     # 循环体
4     statements
5     # 通常需要修改条件相关的变量，避免死循环
```

示例 1：数字反转

Python

```
1 n = int(input("请输入一个整数: ")) # 假设输入 123
2 original = n
3 rev = 0
4
5 while n > 0:
6     last_digit = n % 10 # 取最后一位: 123 % 10 = 3
7     rev = rev * 10 + last_digit # 构建反转数: 0*10 + 3 = 3
8     n = n // 10 # 去掉最后一位: 123 // 10 = 12
9
10 print(f"{original}的反转数是: {rev}")
```

执行过程追踪：

循环次数	n	last_digit	rev（更新后）	n（更新后）
初始	123	—	0	—
第 1 次	123	3	3	12
第 2 次	12	2	32	1
第 3 次	1	1	321	0
结束	0	—	—	—

示例 2：猜数字游戏

Python

```
1 import random
2 target = random.randint(1, 100) # 随机生成1-100的数
3 guess = 0
4 attempts = 0
5
6 while guess != target:
7     guess = int(input("猜一个1-100的数字: "))
8     attempts += 1
9
10    if guess < target:
11        print("猜小了! ")
12    elif guess > target:
13        print("猜大了! ")
14    else:
15        print(f"恭喜! 你用了{attempts}次猜对了! ")
```

示例 3：输入验证（无限循环 + break）

Python

```
1 while True: # 无限循环
2     age = input("请输入年龄 (0-120): ")
3     if age.isdigit() and 0 <= int(age) <= 120:
4         break # 输入有效，退出循环
5     else:
6         print("输入无效，请重新输入! ")
```

for 循环

原理：遍历可迭代对象（如列表、字符串、range 对象）中的每个元素。

Python

```
1 # 基本语法
2 for variable in iterable:
3     # 循环体
4     statements
```

range() 函数详解:

Python

```
1 range(5)          # 0, 1, 2, 3, 4 (从0开始, 到5前停止)
2 range(1, 6)       # 1, 2, 3, 4, 5 (指定起止)
3 range(0, 10, 2)   # 0, 2, 4, 6, 8 (指定步长)
4 range(10, 0, -1)  # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 (负步长)
```

for 循环实例:

Python

```
1 # 示例1: 遍历数字
2 for i in range(5):
3     print(i, end=' ')      # 输出: 0 1 2 3 4
4
5 # 示例2: 遍历字符串
6 for ch in "hello":
7     print(ch)              # 逐字符输出: h e l l o
8
9 # 示例3: 累加求和
10 total = 0
11 for i in range(1, 101):   # 1到100
12     total += i
13 print(f"1到100的和: {total}") # 输出: 5050
14
15 # 示例4: 遍历列表
16 scores = [85, 92, 78, 96, 88]
17 for score in scores:
18     print(score)
```

循环控制语句

Python

```
1 # break: 立即退出整个循环
2 for i in range(10):
3     if i == 5:
4         break          # 当 i 等于 5 时退出循环
5     print(i, end=' ')  # 输出: 0 1 2 3 4
6
7 # continue: 跳过当前迭代, 继续下一次循环
8 for i in range(5):
9     if i == 2:
10        continue       # 跳过 i=2 这次循环
11    print(i, end=' ')   # 输出: 0 1 3 4
12
13 # else 子句: 循环正常结束 (非 break 退出) 时执行
14 for i in range(5):
15     if i == 10:        # 这个条件永远不会满足
16         break
17 else:
18     print("循环正常结束") # 会执行
```

循环嵌套

Python

```
1 # 打印九九乘法表
2 for i in range(1, 10):      # 外层循环: 行
3     for j in range(1, i + 1): # 内层循环: 列
4         print(f"{j}×{i}={i*j:2d}", end=' ')
5     print()                 # 换行
6
7 # 输出:
8 # 1×1= 1
9 # 1×2= 2  2×2= 4
10 # 1×3= 3  2×3= 6  3×3= 9
11 # ...
```

单元练习 3.2

练习1：用 while 循环计算 $1 \times 2 \times 3 \times \dots \times n$ （即 n 的阶乘）。

输入：5

输出：5! = 120

练习2：输入一个整数 n ，用 for 循环输出如下图案（ $n=5$ 时）：

```
*  
**  
***  
****  
*****
```

练习3：找出 100 以内所有能被 3 整除但不能被 5 整除的数，每行输出 5 个。

第四单元：数据结构与容器类型

4.1 列表（List）

创建与基本操作

Python

```
1 # 创建列表
2 empty_list = []           # 空列表
3 numbers = [1, 2, 3, 4, 5] # 整数列表
4 mixed = [1, "hello", 3.14, True] # 混合类型列表
5 nested = [[1, 2], [3, 4]] # 嵌套列表
6
7 # 访问元素
8 nums = [10, 20, 30, 40, 50]
9 print(nums[0])           # 第一个元素: 10 (索引从0开始)
10 print(nums[-1])         # 最后一个元素: 50 (负索引从-1开始)
11 print(nums[-2])         # 倒数第二个: 40
12
13 # 列表切片
14 print(nums[1:3])         # 索引1到2: [20, 30] (左闭右开)
15 print(nums[:3])          # 前3个元素: [10, 20, 30]
16 print(nums[2:])          # 从索引2开始: [30, 40, 50]
17 print(nums[::2])         # 步长为2: [10, 30, 50]
18 print(nums[::-1])        # 反转列表: [50, 40, 30, 20, 10]
```

增删改查

Python

```
1 # 添加元素
2 nums = [1, 2, 3]
3 nums.append(4)           # 末尾添加: [1, 2, 3, 4]
4 nums.insert(1, 1.5)      # 指定位置插入: [1, 1.5, 2, 3, 4]
5 nums.extend([5, 6])     # 扩展列表: [1, 1.5, 2, 3, 4, 5, 6]
6
7 # 删除元素
8 nums = [1, 2, 3, 2, 4]
9 nums.remove(2)           # 删除第一个匹配项: [1, 3, 2, 4]
10 popped = nums.pop()     # 删除并返回最后一个元素: 4, nums变为[1,
    3, 2]
11 popped = nums.pop(1)    # 删除指定位置元素: 3, nums变为[1, 2]
12 del nums[0]             # 删除索引0的元素, nums变为[2]
13
14 # 修改元素
15 nums = [10, 20, 30]
16 nums[1] = 200           # 修改后: [10, 200, 30]
17
18 # 查找元素
19 nums = [3, 1, 4, 1, 5]
20 print(nums.index(1))     # 第一个1的索引: 1
21 print(nums.count(1))     # 元素出现次数: 2
22 print(3 in nums)        # 成员判断: True
```

排序与反转

Python

```
1 nums = [3, 1, 4, 1, 5]
2
3 # 排序（原地修改）
4 nums.sort()                # 升序: [1, 1, 3, 4, 5]
5 nums.sort(reverse=True)    # 降序: [5, 4, 3, 1, 1]
6
7 # 排序（返回新列表，不修改原列表）
8 sorted_nums = sorted(nums) # 升序
9 sorted_nums = sorted(nums, reverse=True) # 降序
10
11 # 反转
12 nums.reverse()            # 原地反转
13 reversed_nums = list(reversed(nums)) # 返回新列表
14
15 # 长度
16 print(len(nums))          # 5
```

列表推导式

Python

```
1 # 传统写法
2 squares = []
3 for i in range(1, 11):
4     squares.append(i ** 2)
5
6 # 列表推导式（更简洁）
7 squares = [i ** 2 for i in range(1, 11)]
8 # 结果: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
9
10 # 带条件的列表推导式
11 even_squares = [i ** 2 for i in range(1, 11) if i % 2 == 0]
12 # 结果: [4, 16, 36, 64, 100]
13
14 # 字符串处理
15 words = ["hello", "world", "python"]
16 upper_words = [w.upper() for w in words]
17 # 结果: ["HELLO", "WORLD", "PYTHON"]
```

4.2 元组 (Tuple)

定义与解包

Python

```
1 # 创建元组 (不可变序列)
2 t1 = (1, 2, 3)
3 t2 = ("a", "b", "c")
4 single = (42,)          # 单元素元组需要逗号!
5 empty = ()
6
7 # 访问元素 (和列表一样)
8 print(t1[0])            # 1
9 print(t1[1:3])          # (2, 3)
10
11 # ❌ 不能修改
12 # t1[0] = 100          # TypeError!
13
14 # 元组解包
15 x, y, z = (10, 20, 30)
16 print(x, y, z)          # 10 20 30
17
18 # 实际应用: 交换变量
19 a, b = 1, 2
20 a, b = b, a              # 一行搞定交换
21
22 # 函数返回多个值 (本质是元组)
23 def get_min_max(lst):
24     return min(lst), max(lst)
25
26 minimum, maximum = get_min_max([5, 2, 8, 1, 9])
```

适用场景：数据不应被修改时（如配置项、坐标点）、函数返回多个值、作为字典的键。

4.3 字典 (Dictionary)

键值对存储

Python

```
1 # 创建字典
2 student = {
3     "name": "Alice",
4     "age": 20,
5     "grade": 95
6 }
7 empty_dict = {}
8
9 # 增删改查
10 print(student["name"])      # 访问: Alice
11 student["age"] = 21        # 修改
12 student["email"] = "a@test.com" # 新增
13 del student["grade"]       # 删除
14
15 # 安全访问
16 print(student.get("phone", "未填写")) # 不存在时返回默认值
17
18 # 遍历
19 for key in student:
20     print(f"{key}: {student[key]}")
21
22 for key, value in student.items():
23     print(f"{key}: {value}")
24
25 for value in student.values():
26     print(value)
```

4.4 集合 (Set)

Python

```
1 # 创建集合（自动去重）
2 s1 = {1, 2, 3, 4, 5}
3 s2 = {4, 5, 6, 7, 8}
4 s3 = set([1, 1, 2, 2, 3])      # 从列表创建：{1, 2, 3}
5
6 # 集合运算
7 print(s1 | s2)                # 并集：{1, 2, 3, 4, 5, 6, 7, 8}
8 print(s1 & s2)                # 交集：{4, 5}
9 print(s1 - s2)                # 差集：{1, 2, 3}
10 print(s1 ^ s2)               # 对称差集：{1, 2, 3, 6, 7, 8}
11
12 # 常用操作
13 s1.add(6)                     # 添加元素
14 s1.discard(3)                 # 删除元素（不存在不报错）
15 print(len(s1))               # 大小
16 print(3 in s1)               # 成员判断
```

4.5 字符串进阶

Python

```
1 text = "Hello, Python World"
2
3 # 常用方法
4 print(text.split())           # 分割: ['Hello,', 'Python', 'World']
5 print(text.split(", "))      # 指定分隔符: ['Hello', 'Python Worl
    d']
6 print("-".join(["a", "b", "c"])) # 连接: 'a-b-c'
7 print(text.replace("World", "世界")) # 替换
8 print(text.find("Python"))     # 查找: 7 (返回索引, -1表示没找到)
9 print(text.upper())           # 转大写
10 print(text.lower())          # 转小写
11 print(text.strip())          # 去除首尾空白
12
13 # 格式化输出
14 name = "Bob"
15 score = 95.678
16 print(f"姓名: {name}, 成绩: {score:.1f}") # 姓名: Bob, 成绩: 95.7
17 print(f"{100:>10}")                # 右对齐, 宽度10: '      100'
    0'
18 print(f"{3.14159:<10.2f}")          # 左对齐, 宽度10: '3.14      '
```

4.6 map() 与 split() 深度解析

map() 函数

Python

```
1 # 原理：将一个函数应用到可迭代对象的每个元素
2 # 语法：map(function, iterable)
3
4 # 示例1：字符串列表 → 整数列表
5 str_list = ['1', '2', '3', '4']
6 int_list = list(map(int, str_list)) # [1, 2, 3, 4]
7
8 # 示例2：计算平方
9 numbers = [1, 2, 3, 4]
10 squares = list(map(lambda x: x**2, numbers)) # [1, 4, 9, 16]
11
12 # 示例3：多个可迭代对象
13 a = [1, 2, 3]
14 b = [4, 5, 6]
15 result = list(map(lambda x, y: x + y, a, b)) # [5, 7, 9]
16
17 # ★ 最常用场景：一行读入多个整数
18 a, b, c = map(int, input().split()) # 输入"10 20 30"
19 # a=10, b=20, c=30
```

split() 函数

Python

```
1 # 语法: str.split(sep=None, maxsplit=-1)
2
3 text = "hello world python"
4 print(text.split())           # 默认按空白分割: ['hello', 'world', 'python']
5
6 text = "apple,banana,orange"
7 print(text.split(','))        # 指定分隔符: ['apple', 'banana', 'orange']
8
9 text = "one two three four"
10 print(text.split(' ', 2))     # 限制分割次数: ['one', 'two', 'three four']
11
12 # 输入处理典型用法
13 input_data = "10 20 30"
14 numbers = list(map(int, input_data.split())) # [10, 20, 30]
```

单元练习 4.1

练习1: 输入一行用空格分隔的若干整数，输出它们的和与平均值（保留2位小数）。

输入: 10 20 30 40

输出: 和=100, 平均值=25.00

练习2: 输入一个字符串，统计其中每个字符出现的次数（用字典实现）。

输入: hello

输出: h:1 e:1 l:2 o:1

练习3: 输入5个学生的姓名和成绩，用字典存储，然后输出最高分学生的姓名和成绩。

第五单元：函数与模块化

5.1 函数定义与调用

基本语法

Python

```
1 def function_name(parameters):
2     """文档字符串（可选）：描述函数功能"""
3     # 函数体
4     return value # 返回值（可选）
```

参数类型

Python

```
1 # 位置参数
2 def add(a, b):
3     return a + b
4 print(add(3, 5)) # 8
5
6 # 默认参数
7 def greet(name, greeting="你好"):
8     return f"{greeting}, {name}!"
9 print(greet("Alice")) # 你好, Alice!
10 print(greet("Bob", "Hello")) # Hello, Bob!
11
12 # 关键字参数
13 def create_profile(name, age, city):
14     return f"{name}, {age}岁, 来自{city}"
15 print(create_profile(age=20, name="Alice", city="深圳"))
16
17 # 可变参数 *args（接收任意数量的位置参数）
18 def calculate_sum(*numbers):
19     return sum(numbers)
20 print(calculate_sum(1, 2, 3, 4, 5)) # 15
21
22 # 可变参数 **kwargs（接收任意数量的关键字参数）
23 def print_info(**kwargs):
24     for key, value in kwargs.items():
25         print(f"{key}: {value}")
26 print_info(name="Alice", age=20, grade="A")
```

返回值

Python

```
1 # 返回单个值
2 def square(x):
3     return x ** 2
4
5 # 返回多个值（本质是元组）
6 def analyze(numbers):
7     return min(numbers), max(numbers), sum(numbers) / len(numbers)
8
9 minimum, maximum, average = analyze([5, 2, 8, 1, 9])
10 print(f"最小={minimum}, 最大={maximum}, 平均={average:.1f}")
```

作用域

Python

```
1 global_var = "我是全局变量"
2
3 def scope_demo():
4     local_var = "我是局部变量"
5     print(global_var)          # 可以读取全局变量
6     # print(local_var)         # 在外面无法访问
7
8 scope_demo()
9 # print(local_var)             # NameError!
10
11 # 修改全局变量需要 global 关键字
12 counter = 0
13
14 def increment():
15     global counter
16     counter += 1
17
18 increment()
19 print(counter)                # 1
```

5.2 高阶函数与 lambda

Python

```
1 # lambda 表达式: 匿名函数
2 square = lambda x: x ** 2
3 print(square(5))                # 25
4
5 # 函数作为参数
6 def apply_operation(nums, operation):
7     return [operation(x) for x in nums]
8
9 result = apply_operation([1, 2, 3, 4], lambda x: x * 10)
10 print(result)                  # [10, 20, 30, 40]
11
12 # 常用高阶函数
13 numbers = [1, 2, 3, 4, 5]
14
15 # filter: 过滤
16 evens = list(filter(lambda x: x % 2 == 0, numbers)) # [2, 4]
17
18 # sorted + key
19 students = [("Alice", 95), ("Bob", 82), ("Charlie", 88)]
20 by_score = sorted(students, key=lambda s: s[1], reverse=True)
21 # [('Alice', 95), ('Charlie', 88), ('Bob', 82)]
```

5.3 模块与包

Python

```
1 # 导入模块
2 import math
3 print(math.sqrt(16))          # 4.0
4 print(math.pi)               # 3.141592653589793
5
6 # 导入指定内容
7 from math import sqrt, pi
8 print(sqrt(25))              # 5.0
9
10 # 导入并重命名
11 import numpy as np
12
13 # 自定义模块
14 # 将常用函数保存为 my_utils.py, 然后在其他文件中:
15 # import my_utils
16 # my_utils.my_function()
```

单元练习 5.1

练习：编写一个函数 `is_prime(n)`，判断一个整数是否为素数。再用这个函数找出 2 到 100 之间所有的素数。

输出示例： 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83
89 97

第六单元：文件与异常处理

6.1 文件读写

Python

```
1 # 写入文件
2 with open("output.txt", "w", encoding="utf-8") as f:
3     f.write("第一行内容\n")
4     f.write("第二行内容\n")
5
6 # 读取文件
7 with open("output.txt", "r", encoding="utf-8") as f:
8     content = f.read()          # 读取全部内容
9     print(content)
10
11 # 逐行读取（推荐，内存友好）
12 with open("output.txt", "r", encoding="utf-8") as f:
13     for line in f:
14         print(line.strip())     # strip() 去掉换行符
15
16 # 追加内容
17 with open("output.txt", "a", encoding="utf-8") as f:
18     f.write("新追加的一行\n")
```

文件打开模式：

模式	含义
<code>"r"</code>	只读（默认）
<code>"w"</code>	写入（覆盖原有内容）
<code>"a"</code>	追加（在末尾添加）
<code>"rb"</code> / <code>"wb"</code>	二进制读写

 始终使用 `with` 语句：它会自动关闭文件，即使发生异常也能安全处理。

6.2 异常处理

Python

```
1 # 基本结构
2 try:
3     # 可能出错的代码
4     result = 10 / 0
5 except ZeroDivisionError:
6     # 处理特定异常
7     print("除数不能为零！")
8 finally:
9     # 无论是否异常都会执行
10    print("计算结束")
11
12 # 捕获多种异常
13 try:
14     num = int(input("请输入数字："))
15     result = 100 / num
16 except ValueError:
17     print("输入的不是有效数字！")
18 except ZeroDivisionError:
19     print("不能输入零！")
20
21 # 捕获所有异常（不推荐，仅用于调试）
22 try:
23     risky_operation()
24 except Exception as e:
25     print(f"出错了：{e}")
26
27 # 主动抛出异常
28 def validate_age(age):
29     if age < 0 or age > 150:
30         raise ValueError("年龄必须在0-150之间")
31     return age
```

单元练习 6.1

练习：编写一个程序，尝试打开并读取一个文件。如果文件不存在，提示用户并创建该文件；如果文件存在，统计其中的行数和字符数。

基础掌握模块 · 课后作业

作业1：编写一个温度转换器，输入摄氏温度，输出华氏温度和开尔文温度。

华氏度 = 摄氏度 $\times$ 9/5 + 32

开尔文 = 摄氏度 + 273.15

作业2：实现简易计算器，输入两个数字和运算符（+、-、*、/），输出计算结果。处理除零错误。

作业3：输入一个字符串，判断它是否是回文（正读反读一样，如"level"）。

作业4：用列表存储10个学生的成绩，实现：① 最高分 ② 最低分 ③ 平均分 ④ 及格率（ ≥ 60 为及格）。

作业5：编写一个函数 `bubble_sort(lst)`，对列表进行冒泡排序（不要使用内置的 `sort()`）。

第二部分：竞赛进阶

第五单元：排序算法

5.1 冒泡排序（Bubble Sort）

算法原理

核心思想：相邻元素两两比较，如果顺序错误就交换。每一轮会将当前最大的元素"冒泡"到末尾。

执行过程演示（排序 `[5, 3, 8, 1, 2]`）：

Plain Text

- 1 初始： [5, 3, 8, 1, 2]
- 2 第1轮：比较交换后 → [3, 5, 1, 2, 8] （8冒到最后）
- 3 第2轮：比较交换后 → [3, 1, 2, 5, 8] （5冒到倒数第二）
- 4 第3轮：比较交换后 → [1, 2, 3, 5, 8] （3冒到正确位置）
- 5 第4轮：无交换 → [1, 2, 3, 5, 8] （已有序，可提前结束）

代码实现

Python

```
1 def bubble_sort(nums):
2     """冒泡排序：原地排序，时间复杂度  $O(n^2)$ """
3     n = len(nums)
4     for i in range(n - 1):          # 需要 n-1 轮
5         swapped = False             # 优化：标记是否发生交换
6         for j in range(n - 1 - i):  # 每轮比较次数递减
7             if nums[j] > nums[j + 1]: # 前一个比后一个大，交换
8                 nums[j], nums[j + 1] = nums[j + 1], nums[j]
9                 swapped = True
10        if not swapped:              # 没有交换说明已经有序
11            break
12    return nums
13
14 # 测试
15 data = [5, 3, 8, 1, 2]
16 print(bubble_sort(data.copy()))    # [1, 2, 3, 5, 8]
```

复杂度分析

指标	值
时间复杂度（最坏）	$O(n^2)$
时间复杂度（最好）	$O(n)$ （已排序时，加优化）
空间复杂度	$O(1)$ （原地排序）
稳定性	✅ 稳定（相等元素不交换）

Example 5.1

题目：输入 n 个整数，用冒泡排序从小到大排序后输出。

输入：

Plain Text

```
1 5
2 5 3 8 1 2
```

输出：

Plain Text

```
1 1 2 3 5 8
```

完整解答：

Python

```
1 n = int(input())
2 nums = list(map(int, input().split()))
3
4 for i in range(n - 1):
5     for j in range(n - 1 - i):
6         if nums[j] > nums[j + 1]:
7             nums[j], nums[j + 1] = nums[j + 1], nums[j]
8
9 print(*nums) # * 解包输出，自动用空格分隔
```

5.2 桶排序（Bucket Sort / Counting Sort）

算法原理

核心思想：创建一个"桶数组"，遍历原始数据，将每个元素放入对应编号的桶中（计数），最后按桶的顺序依次取出。

适用场景：数据范围已知且不太大（如 0~100 的成绩排序）。

执行过程演示（排序 `[50, 30, 80, 10, 20]`）：

Plain Text

```
1 创建101个桶 (0~100)，初始全为0
2 ↓
3 遍历数据：
4   50 → 桶[50]++
5   30 → 桶[30]++
6   80 → 桶[80]++
7   10 → 桶[10]++
8   20 → 桶[20]++
9 ↓
10 桶状态：桶[10]=1，桶[20]=1，桶[30]=1，桶[50]=1，桶[80]=1
11 ↓
12 从桶[0]到桶[100]依次输出：
13   10, 20, 30, 50, 80
```

代码实现

Python

```
1 def bucket_sort(nums, max_val=100):
2     """桶排序/计数排序：时间复杂度 O(n + k)"""
3     buckets = [0] * (max_val + 1)    # 创建桶
4
5     for num in nums:                  # 计数阶段
6         buckets[num] += 1
7
8     result = []                       # 输出阶段
9     for i in range(max_val + 1):
10         for j in range(buckets[i]): # 桶中有几个就输出几个
11             result.append(i)
12
13     return result
14
15 # 测试
16 data = [50, 30, 80, 10, 20]
17 print(bucket_sort(data))             # [10, 20, 30, 50, 80]
```

复杂度分析

指标	值
时间复杂度	$O(n + k)$, k 为桶的数量
空间复杂度	$O(k)$
稳定性	 稳定

Example 5.2

题目：输入 n 个 $0\sim100$ 之间的整数，用桶排序从小到大排序后输出。

输入：

Plain Text

1 5
2 50 30 80 10 20

输出：

Plain Text

1 10 20 30 50 80

完整解答：

Python

```
1 n = int(input())
2 nums = list(map(int, input().split()))
3
4 buckets = [0] * 101
5
6 for num in nums:
7     buckets[num] += 1
8
9 result = []
10 for i in range(101):
11     for j in range(buckets[i]):
12         result.append(i)
13
14 print(*result)
```

单元练习 5.1（课堂练习）

练习1：用冒泡排序对字符串数组按长度从小到大排序。

输入：apple banana pear peach grape

输出：pear apple grape peach banana

练习2：有 1000000 个 0~9 之间的随机数，用桶排序统计每个数字出现的次数，按数字从小到大输出。

课后作业 5

作业1：实现冒泡排序的降序版本（从大到小排序）。

作业2：输入 n 个学生的姓名和成绩（成绩 0~100），用桶排序按成绩从高到低输出学生姓名。

输入示例：

```
5
Alice 85
Bob 92
Charlie 78
David 92
Eve 85
```

输出示例（成绩相同按输入顺序）：
Bob David Alice Eve Charlie

第六单元：深度优先搜索（DFS）

6.1 DFS 基本原理

核心思想：一条路走到黑，走不通就回退一步，换条路继续走。类似走迷宫时"一路向右手边走"的策略。

与 BFS 的对比：

特性	DFS	BFS
数据结构	栈（递归 / 显式栈）	队列
空间复杂度	$O(\text{深度})$	$O(\text{宽度})$
适合场景	找所有解、路径搜索	找最短路径
实现方式	递归（最自然）	队列循环

6.2 全排列问题

Example 6.1：生成全排列

题目：输入 n ，输出 $1\sim n$ 的所有排列。

输入： 3

输出：

Plain Text

```
1 1 2 3
2 1 3 2
3 2 1 3
4 2 3 1
5 3 1 2
6 3 2 1
```

完整解答：

Python

```
1 def dfs(path, used, n):
2     """DFS 生成全排列"""
3     if len(path) == n:                # 终止条件：路径长度等于n
4         print(*path)
5         return
6
7     for i in range(1, n + 1):
8         if not used[i]:                # 如果数字 i 还没被使用
9             used[i] = True              # 标记为已使用
10            path.append(i)               # 加入路径
11            dfs(path, used, n)           # 递归搜索
12            path.pop()                   # 回溯：撤销选择
13            used[i] = False              # 回溯：标记为未使用
14
15 n = int(input())
16 used = [False] * (n + 1)
17 dfs([], used, n)
```

执行过程追踪 (n=3)：

Plain Text

```
1 dfs([], used, 3)
2 └─ i=1: used[1]=T, path=[1]
3   └─ i=2: used[2]=T, path=[1,2]
4     └─ i=3: used[3]=T, path=[1,2,3] → 输出!
5     └─ 回溯: path=[1,2], used[3]=F
6     └─ i=3: used[3]=T, path=[1,3]
7     └─ i=2: used[2]=T, path=[1,3,2] → 输出!
8     └─ 回溯
9 └─ i=2: ...
10 └─ i=3: ...
```

关键三步骤 (DFS 模板)：

1. 选择：标记已访问，加入路径

2. **递归**：深入下一层
3. **回溯**：撤销选择，恢复状态

Example 6.2：迷宫问题

题目：在 $n \times m$ 的迷宫中，从起点 $(0,0)$ 到终点 $(n-1, m-1)$ ，只能上下左右走，求是否存在路径。

完整解答：

Python

```
1 def dfs(x, y):
2     """DFS 搜索迷宫路径"""
3     if x == n - 1 and y == m - 1:    # 到达终点
4         return True
5
6     visited[x][y] = True              # 标记为已访问
7
8     # 四个方向: 上、右、下、左
9     directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]
10
11     for dx, dy in directions:
12         nx, ny = x + dx, y + dy
13         # 检查边界、可通行、未访问
14         if (0 <= nx < n and 0 <= ny < m
15             and maze[nx][ny] == 0 and not visited[nx][ny]):
16             if dfs(nx, ny):
17                 return True
18
19     visited[x][y] = False             # 回溯
20     return False
21
22 # 输入
23 n, m = map(int, input().split())
24 maze = []
25 for i in range(n):
26     maze.append(list(map(int, input().split())))
27
28 visited = [[False] * m for _ in range(n)]
29
30 if dfs(0, 0):
31     print("有路径! ")
32 else:
33     print("无路径! ")
```

单元练习 6.1 (课堂练习)

练习1: 输入 n , 输出 $1 \sim n$ 中所有和为 n 的组合 (每个数只能用一次, 顺序不同算同一种)。

输入:

输出：

```
1 4
2 3
1 2 2 (不允许，每个数只能用一次)
5
```

练习2：在 $n \times n$ 的棋盘上放置 n 个皇后，使它们互不攻击（不在同一行、列、对角线）。输出一种可行方案。

课后作业 6

作业1：用 DFS 解决"岛屿数量"问题。给定一个 0/1 矩阵，1 表示陆地，0 表示水，计算岛屿数量（上下左右相连的陆地算一个岛）。

输入：

```
4 4
1 1 0 0
1 0 0 1
0 0 1 1
0 0 1 0
输出：3
```

作业2：用 DFS 找出图中从起点到终点的所有路径。

第七单元：广度优先搜索（BFS）

7.1 BFS 基本原理

核心思想：从起点开始，先访问所有距离为 1 的点，再访问距离为 2 的点……逐层扩展。类似水波扩散。

实现方式：队列（Queue）

7.2 BFS 模板

Python

```
1 from collections import deque
2
3 def bfs(start, target):
4     queue = deque([start])          # 队列初始化
5     visited = {start}               # 已访问集合
6     distance = {start: 0}           # 距离记录
7
8     while queue:
9         current = queue.popleft()    # 取出队首
10
11         if current == target:        # 找到目标
12             return distance[current]
13
14         for neighbor in get_neighbors(current):
15             if neighbor not in visited:
16                 visited.add(neighbor)
17                 distance[neighbor] = distance[current] + 1
18                 queue.append(neighbor)
19
20     return -1 # 找不到
```

7.3 Example 7.1: 最短路径（迷宫）

题目：在 $n \times m$ 的迷宫中，0 表示通路，1 表示墙壁，求从 (0,0) 到 (n-1,m-1) 的最短步数。

输入：

Plain Text

```
1 5 5
2 0 1 0 0 0
3 0 1 0 1 0
4 0 0 0 1 0
5 1 1 0 1 0
6 0 0 0 0 0
```

输出：

完整解答：

Python

```
1 from collections import deque
2
3 def bfs():
4     queue = deque([(0, 0, 0)])      # (x, y, steps)
5     visited = [[False] * m for _ in range(n)]
6     visited[0][0] = True
7
8     # 四个方向
9     directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]
10
11     while queue:
12         x, y, steps = queue.popleft()
13
14         # 到达终点
15         if x == n - 1 and y == m - 1:
16             return steps
17
18         for dx, dy in directions:
19             nx, ny = x + dx, y + dy
20             if (0 <= nx < n and 0 <= ny < m
21                 and maze[nx][ny] == 0 and not visited[nx][ny]):
22                 visited[nx][ny] = True
23                 queue.append((nx, ny, steps + 1))
24
25     return -1 # 无法到达
26
27 # 输入
28 n, m = map(int, input().split())
29 maze = []
30 for i in range(n):
31     maze.append(list(map(int, input().split())))
32
33 result = bfs()
34 print(result)
```

BFS 逐层扩展示意图：

Plain Text

```
1 初始: S . . . .
2      . # . # .
3      . . . # .
4      # # . # .
5      . . . . E
6
7 第0层: S
8 第1层: S → (1,0)
9 第2层: (1,0) → (2,0) → (2,1)
10 第3层: (2,1) → (2,2)
11 ...
12 最短路径找到 E
```

7.4 Example 7.2: 洪水填充 (Flood Fill)

题目：给定一个矩阵，从指定起点开始，将与起点连通的所有相同值区域标记为新值。

完整解答：

```
1 from collections import deque
2
3 def flood_fill(grid, sr, sc, new_color):
4     """洪水填充: 将与(sr,sc)连通的同色区域改为new_color"""
5     rows, cols = len(grid), len(grid[0])
6     original_color = grid[sr][sc]
7
8     if original_color == new_color:
9         return grid
10
11     queue = deque([(sr, sc)])
12     grid[sr][sc] = new_color
13
14     directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]
15
16     while queue:
17         r, c = queue.popleft()
18         for dr, dc in directions:
19             nr, nc = r + dr, c + dc
20             if (0 <= nr < rows and 0 <= nc < cols
21                 and grid[nr][nc] == original_color):
22                 grid[nr][nc] = new_color
23                 queue.append((nr, nc))
24
25     return grid
```

单元练习 7.1 (课堂练习)

练习1: 在一个 8×8 的棋盘上, 马 (骑士) 从 (0,0) 出发, 求到达 (7,7) 的最少步数。(马走"日"字)

练习2: 给定一个字符串和一本"字典" (字符串列表), 每次可以变换字符串中的一个字符, 且变换后的字符串必须在字典中。求从起始字符串到目标字符串的最短变换序列长度。

课后作业 7

作业1: 在 $n \times m$ 的网格中, 有些格子有障碍物。从 (0,0) 出发, 每次可以上下左右走, 求到达 (n-1,m-1) 的最少步数。如果无法到达, 输出 -1。

作业2：用 BFS 解决"水壶问题"：有两个容量分别为 a 和 b 的水壶，求能否量出恰好 c 升水。

第八单元：递归

8.1 递归基本原理

核心思想：函数调用自身来解决问题。大问题分解为同类小问题。

递归三要素：

1. 终止条件 (Base Case)：什么时候不再递归
2. 递归调用：如何把问题分解为更小的子问题
3. 返回结果：如何将子问题的结果组合成最终答案

8.2 经典递归问题

Example 8.1：斐波那契数列

题目：求斐波那契数列的第 n 项 ($F(1)=1, F(2)=1, F(n)=F(n-1)+F(n-2)$)。

方法一：朴素递归（理解原理，效率低）

Python

```
1 def fib(n):
2     """斐波那契数列 - 递归版"""
3     if n <= 2:                    # 终止条件
4         return 1
5     return fib(n - 1) + fib(n - 2) # 递归调用
6
7 print(fib(10))                  # 55
```

方法二：记忆化递归（优化）

Python

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)
4 def fib_fast(n):
5     if n <= 2:
6         return 1
7     return fib_fast(n - 1) + fib_fast(n - 2)
8
9 print(fib_fast(100))                # 快速计算大数
```

方法三：递推 / 迭代（最高效）

Python

```
1 def fib_iter(n):
2     if n <= 2:
3         return 1
4     a, b = 1, 1
5     for i in range(3, n + 1):
6         a, b = b, a + b
7     return b
8
9 print(fib_iter(100))                # 354224848179261915075
```

Example 8.2：汉诺塔

题目：将 n 个盘子从柱子 A 移动到柱子 C，每次只能移动一个盘子，大盘不能放在小盘上。输出移动步骤。

完整解答：

Python

```
1 def hanoi(n, source, target, auxiliary):
2     """汉诺塔递归解法"""
3     if n == 1:
4         print(f"将盘子1从{source}移到{target}")
5         return
6
7     # 将上面 n-1 个盘子从 source 移到 auxiliary
8     hanoi(n - 1, source, auxiliary, target)
9
10    # 将最大的盘子从 source 移到 target
11    print(f"将盘子{n}从{source}移到{target}")
12
13    # 将 n-1 个盘子从 auxiliary 移到 target
14    hanoi(n - 1, auxiliary, target, source)
15
16 # 测试
17 hanoi(3, 'A', 'C', 'B')
```

输出：

Plain Text

```
1 将盘子1从A移到C
2 将盘子2从A移到B
3 将盘子1从C移到B
4 将盘子3从A移到C
5 将盘子1从B移到A
6 将盘子2从B移到C
7 将盘子1从A移到C
```

递归分解图 (n=3)：

Plain Text

```
1 hanoi(3, A, C, B)
2 |─ hanoi(2, A, B, C)
3 |   |─ hanoi(1, A, C, B) → A→C
4 |   |─ A→B
5 |   |─ hanoi(1, C, B, A) → C→B
6 |─ A→C
7 |─ hanoi(2, B, C, A)
8     |─ hanoi(1, B, A, C) → B→A
9     |─ B→C
10    |─ hanoi(1, A, C, B) → A→C
```

Example 8.3: 二分搜索（递归版）

Python

```
1 def binary_search(arr, target, left, right):
2     """在有序数组 arr 中查找 target"""
3     if left > right:                # 终止条件：找不到
4         return -1
5
6     mid = (left + right) // 2
7
8     if arr[mid] == target:
9         return mid                  # 找到了
10    elif arr[mid] > target:
11        return binary_search(arr, target, left, mid - 1) # 左半部分
12    else:
13        return binary_search(arr, target, mid + 1, right) # 右半部分
14
15 # 测试
16 nums = [1, 3, 5, 7, 9, 11, 13]
17 print(binary_search(nums, 7, 0, len(nums) - 1)) # 3
18 print(binary_search(nums, 8, 0, len(nums) - 1)) # -1
```

单元练习 8.1（课堂练习）

练习1：用递归实现求 n 的阶乘。

输入：5
输出：120

练习2：用递归实现字符串反转。

输入：hello
输出：olleh

练习3：一只青蛙一次可以跳 1 级或 2 级台阶，问跳上 n 级台阶有多少种方法？用递归实现。

课后作业 8

作业1：用递归实现求最大公约数（辗转相除法）。
提示： $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ ，当 $b=0$ 时 $\text{gcd}=a$ 。

作业2：用递归判断一个字符串是否是回文。

作业3：用递归解决"子集生成"问题：给定一个集合，输出它所有的子集。

第九单元：递推

9.1 递推基本原理

核心思想：从已知初始条件出发，通过递推公式逐步计算出后续项。与递归方向相反——递归是从大问题到小问题，递推是从小问题到大问题。

递推 vs 递归：

特性	递推	递归
方向	从小到大	从大到小
实现	循环	函数调用自身
空间	$O(1)$ 或 $O(n)$	$O(n)$ 调用栈
效率	通常更高	可能有重复计算

9.2 经典递推问题

Example 9.1: 爬楼梯

题目：一次可以走 1 级或 2 级台阶，上 n 级台阶有多少种走法？

分析：

- 到达第 n 级，要么从第 $n-1$ 级走 1 步，要么从第 $n-2$ 级走 2 步
- 递推公式： $dp[n] = dp[n-1] + dp[n-2]$
- 初始条件： $dp[1] = 1, dp[2] = 2$

完整解答：

Python

```
1 def climb_stairs(n):
2     """爬楼梯 - 递推解法"""
3     if n <= 2:
4         return n
5
6     a, b = 1, 2          # dp[1]=1, dp[2]=2
7     for i in range(3, n + 1):
8         a, b = b, a + b  # 滚动更新
9
10    return b
11
12 # 测试
13 for i in range(1, 6):
14     print(f"{i}级台阶: {climb_stairs(i)}种")
15
16 # 输出:
17 # 1级台阶: 1种
18 # 2级台阶: 2种
19 # 3级台阶: 3种
20 # 4级台阶: 5种
21 # 5级台阶: 8种
```

Example 9.2: 最大公约数（辗转相除法）

题目：输入两个正整数 a 和 b ，求它们的最大公约数。

算法原理: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ ，直到 $b = 0$ 时 $\text{gcd} = a$ 。

完整解答:

Python

```
1 # 方法一: 循环 (递推)
2 def gcd_iter(a, b):
3     while b != 0:
4         a, b = b, a % b
5     return a
6
7 # 方法二: 递归
8 def gcd_recur(a, b):
9     if b == 0:
10         return a
11     return gcd_recur(b, a % b)
12
13 # 测试
14 print(gcd_iter(48, 18))          # 6
15 print(gcd_recur(48, 18))        # 6
16
17 # 最小公倍数: lcm(a, b) = a * b / gcd(a, b)
18 def lcm(a, b):
19     return a * b // gcd_iter(a, b)
20
21 print(lcm(12, 18))              # 36
```

执行过程追踪 ($\text{gcd}(48, 18)$):

Plain Text

```
1 gcd(48, 18)
2 → 48 % 18 = 12 → gcd(18, 12)
3 → 18 % 12 = 6 → gcd(12, 6)
4 → 12 % 6 = 0 → gcd(6, 0)
5 → 返回 6
```

Example 9.3: 百钱买百鸡

题目：公鸡 5 元 / 只，母鸡 3 元 / 只，小鸡 1 元 / 3 只。用 100 元买 100 只鸡，有多少种方案？

分析：

- 设公鸡 x 只，母鸡 y 只，小鸡 z 只
- 条件 1: $x + y + z = 100$
- 条件 2: $5x + 3y + z/3 = 100$
- 条件 3: z 必须是 3 的倍数

完整解答：

Python

```
1 print("百钱买百鸡的所有方案：")
2 count = 0
3
4 for x in range(0, 21):          # 公鸡最多20只 (5元×20=100元)
5     for y in range(0, 34):      # 母鸡最多33只 (3元×33=99元)
6         z = 100 - x - y        # 小鸡数量
7
8         if 5 * x + 3 * y + z / 3 == 100 and z % 3 == 0:
9             count += 1
10            print(f"方案{count}: 公鸡{x}只, 母鸡{y}只, 小鸡{z}只")
11
12 # 输出：
13 # 方案1: 公鸡0只, 母鸡25只, 小鸡75只
14 # 方案2: 公鸡4只, 母鸡18只, 小鸡78只
15 # 方案3: 公鸡8只, 母鸡11只, 小鸡81只
16 # 方案4: 公鸡12只, 母鸡4只, 小鸡84只
```

Example 9.4: 数字反转

题目：输入一个整数，输出它的反转数。

完整解答：

Python

```
1 n = int(input())                # 假设输入 12345
2 original = n
3 rev = 0
4
5 while n > 0:
6     last_digit = n % 10          # 取最后一位
7     rev = rev * 10 + last_digit  # 构建反转数
8     n //= 10                    # 去掉最后一位
9
10 print(f"{original}的反转数是: {rev}")
11 # 12345 → 54321
```

单元练习 9.1 (课堂练习)

练习1: 用递推方法求 $1! + 2! + 3! + \dots + n!$ 的和。

输入: 5

输出: 153 ($1! + 2! + 3! + 4! + 5! = 1 + 2 + 6 + 24 + 120$)

练习2: 约瑟夫环问题: n 个人围成一圈, 从 1 开始报数, 数到 3 的人出局。求最后剩下的人最初的编号。

输入: 10

输出: 4

课后作业 9

作业1: 用递推实现"杨辉三角"前 n 行。

输入: 5

输出:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

作业2: 用递推解决"铺砖问题": 用 1×1 和 1×2 的砖铺满 $1 \times n$ 的地面, 有多少种铺法?

作业3: 用辗转相除法和更相减损术分别实现最大公约数, 对比两种方法的效率。

竞赛进阶模块 · 综合练习

综合题1： 在一个 $n \times m$ 的网格中，从左上角到右下角，每次只能向右或向下走。求总共有多少种不同的路径？

综合题2： 用 DFS + 剪枝解决"N皇后"问题：在 $n \times n$ 的棋盘上放置 n 个皇后，使它们互不攻击。输出所有可行方案的数量。

综合题3： 给定一个无向图（邻接矩阵表示），用 BFS 判断图是否连通（从任意点出发能到达所有点）。

— 教材正文结束 —

新哲文院算法社 2026-2027 学年